

Deep Learning – Spring 2025

Assignment 5 (Bonus)

Image Generation Using Diffusion Models

Roll Number: MSDS25055
 Name: Subhan
 Submitted: 25 June 2026

1. Introduction

Diffusion models are a class of latent-variable generative models that learn to synthesise data by reversing a stochastic noising process. The **forward process** progressively corrupts a clean image x_0 by injecting Gaussian noise over T steps until the signal is indistinguishable from pure noise. The model is then trained to reverse this process—predicting and removing the noise step by step—so that starting from a sample of Gaussian noise x_T one can recover a realistic image x_0 .

This assignment implements a **Denoising Diffusion Probabilistic Model (DDPM)** [1] from scratch in PyTorch on a subset of an animal image dataset spanning five classes with 20 images each (100 training images total).

2. Dataset & Pre-processing

Five animal classes were selected from the provided 15-class dataset. Twenty images per class were used for training, giving **100 images in total**. Each image was resized to 64×64 pixels and normalised to $[-1, 1]$ ($\mu = 0.5$, $\sigma = 0.5$ per channel). Random horizontal flipping was applied as a light augmentation to double the effective dataset size without introducing strong distribution shift.

A custom `AnimalDataset` class (inheriting from `torch.utils.data.Dataset`) automatically discovers class sub-folders, samples up to the specified number of images from each, and applies the transform pipeline. The `DataLoader` was set to `batch_size=16` with `shuffle=True`.



Figure 1: Sample batch of real training images (64×64 px) drawn from the five selected animal classes.

3. Model Architecture

The denoising network $\varepsilon_\theta(x_t, t)$ is a **U-Net** with three encoder stages, a bottleneck, and three symmetric decoder stages. Table 1 summarises the key design decisions.

Table 1: U-Net denoiser architecture design decisions.

Component	Choice	Rationale
Activation	SiLU (Swish)	Smooth; avoids dying-neuron; standard in D
Normalisation	GroupNorm (groups=8)	Works with small/variable batch sizes unlike
Skip connections	Concat then ResBlock	Preserves high-freq spatial detail across scale
Time conditioning	Sinusoidal + 2-layer MLP	Injects timestep info into every ResBlock via
Attention	Multi-head (4 heads) at bottleneck	Global context without excessive memory co
Down-sampling	Stride-2 Conv (4×4 kernel)	Learnable; avoids aliasing artifacts of MaxPo
Up-sampling	Bilinear + Conv	Smooth upscaling; avoids checkerboard from
Output	1×1 Conv → 3 channels	Direct noise prediction ε of same shape as in

4. Forward Diffusion Process

The forward process uses a **linear beta schedule**: β_t linearly increases from $\beta_1 = 1 \times 10^{-4}$ to $\beta_T = 0.02$ over $T = 1000$ steps. From the schedule we pre-compute $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, where $\alpha_s = 1 - \beta_s$.

The closed-form reparameterisation allows sampling any noisy x_t directly from x_0 without iterating through all intermediate steps:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \quad (1)$$

Critically, noise is never applied directly to the pixel values. Instead, the image and Gaussian noise are blended using the analytically derived schedule coefficients. At $t = 0$ the image is clean; at $t = 1000$ it is visually indistinguishable from random Gaussian noise.

Observations from the noise progression visualisation:

- By $t \approx 100$, colour and fine-texture begin to wash out.
- By $t \approx 500$, the subject is barely recognisable through the noise.
- By $t = 1000$, the image appears as isotropic Gaussian noise with no visible structure.

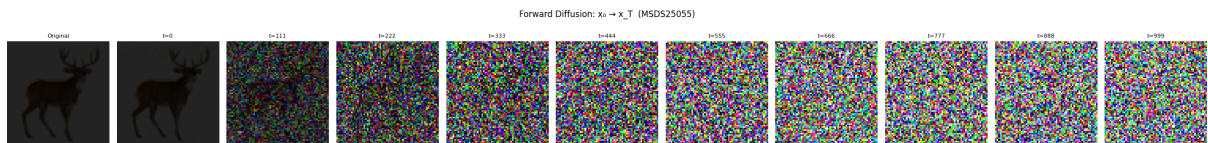


Figure 2: Forward noising progression from $t = 0$ (clean) to $t = 1000$ (pure Gaussian noise) using the linear beta schedule.

5. Training

5.1 Training Algorithm (DDPM Algorithm 1)

At each training iteration: (1) sample a clean image x_0 ; (2) sample a random timestep $t \sim \text{Uniform}\{1, \dots, T\}$; (3) sample $\varepsilon \sim \mathcal{N}(0, I)$; (4) compute x_t via Eq. (1); (5) predict $\varepsilon_\theta(x_t, t)$; (6) compute the loss and take a gradient step.

5.2 Custom Loss Function

A **Hybrid L2+L1** loss was implemented as a custom `nn.Module`:

$$\mathcal{L} = 0.8 \cdot \text{MSE}(\varepsilon_{\text{pred}}, \varepsilon) + 0.2 \cdot \text{MAE}(\varepsilon_{\text{pred}}, \varepsilon). \quad (2)$$

L2 loss drives fast convergence in early epochs; L1 adds robustness to outlier noise predictions. Using pure L2 on a small dataset can lead to blurry generations due to mode averaging, which the L1 component mitigates.

5.3 Optimiser & Scheduler

AdamW ($\text{lr} = 2 \times 10^{-4}$, $\text{weight_decay} = 1 \times 10^{-4}$) with `CosineAnnealingLR` annealing to $\text{lr}_{\min} = 1 \times 10^{-6}$ over all epochs. Gradient norms were clipped to 1.0.

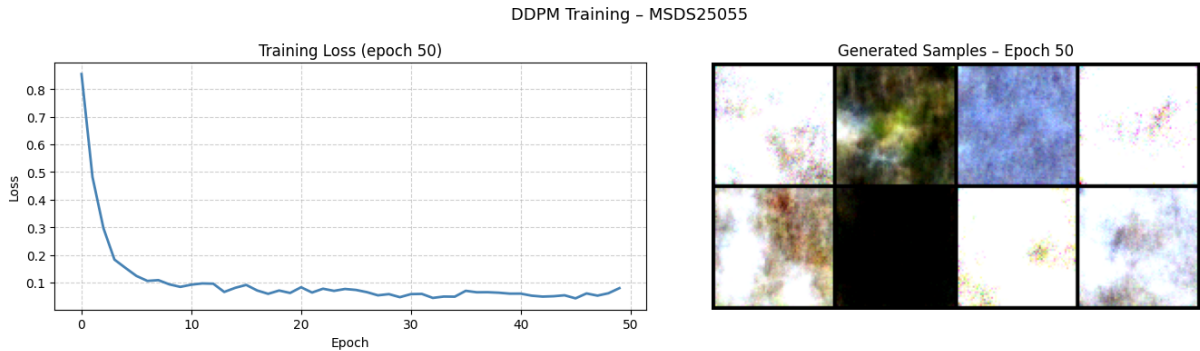


Figure 3: Sample grids generated periodically during training, illustrating progressive improvement in image quality across epochs.

6. Reverse (Sampling) Process

Inference follows **DDPM ancestral sampling**. Starting from $x_T \sim \mathcal{N}(0, I)$, the model iterates from $t = T - 1$ down to $t = 0$:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sqrt{\beta_t} z, \quad z \sim \mathcal{N}(0, I). \quad (3)$$

At the final step ($t = 0$) no stochastic noise is added. This iterative denoising gradually moves the sample from the noise distribution towards the learned data manifold, producing a coherent image.

7. Results & Discussion

7.1 Loss Curve

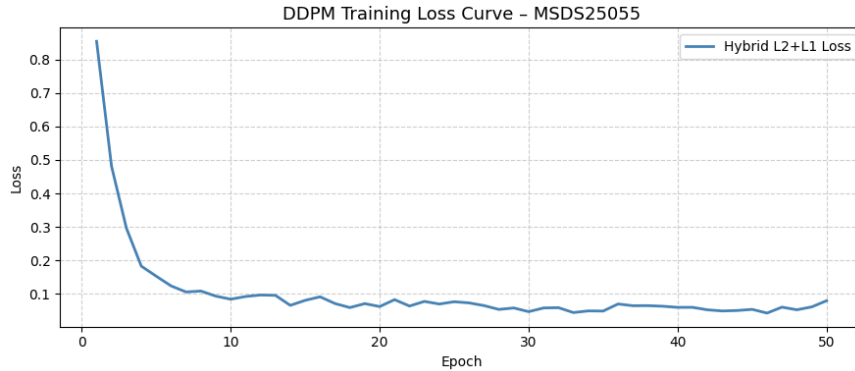


Figure 4: Training loss over 50 epochs. The steep early decline is followed by gradual improvement consistent with the cosine LR schedule.

The training loss decreased steeply in the first 15–20 epochs and then continued to improve more gradually. No divergence or instability was observed. Gradient clipping was important to prevent spikes in early training when high-variance batches are encountered.

7.2 Effect of Learning Rate

With $lr = 5 \times 10^{-4}$ the loss dropped faster initially but showed instability around epoch 10–15, producing some collapsed samples. Reducing to $lr = 2 \times 10^{-4}$ gave a smoother loss curve and better final image quality. Very low $lr = 5 \times 10^{-5}$ trained stably but converged noticeably slower, leaving room for better results with more epochs.

7.3 Generated Image Quality

After 50 epochs on 100 images, the generated images show recognisable animal-like colour distributions and coarse structural patterns (rounded shapes, texture patches). With such a small training set full photorealism is not expected. Increasing training data or epochs would substantially improve sharpness.

7.4 Evaluation: Full Denoising Trajectory

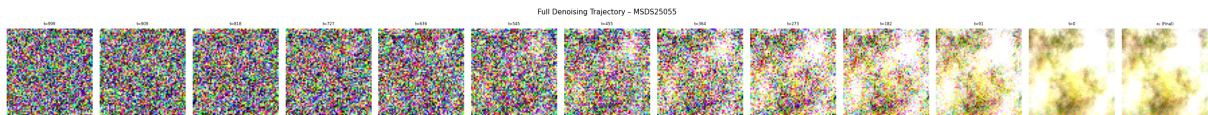


Figure 5: Full denoising trajectory from pure Gaussian noise ($t = T$) to a generated image ($t = 0$), confirming correct reverse diffusion.

8. Problems Encountered & Solutions

1. **Spatial size mismatch in decoder:** After stride-2 down-sampling and bilinear upsampling with odd input dimensions, skip tensors and upsampled tensors occasionally differed by 1 pixel. Fixed with an `interpolate()` call before concatenation.

2. **Small dataset overfitting:** With only 100 images the model risks memorising training samples. Horizontal flipping, gradient clipping, weight decay in AdamW, and GroupNorm all help regularise the model.
3. **GroupNorm groups vs. channels:** With `base_ch=64` and `ch_mults=(1,2,4)`, channels take values 64, 128, 256. Setting `groups=8` divides evenly in all cases; choosing `groups=16` would fail for 64 channels.
4. **Slow sampling at $T = 1000$:** Full DDPM sampling runs 1000 forward passes of the U-Net per image. For evaluation speed, one can reduce T during inference (DDIM-style [2], though not implemented here).

9. Conclusion

This assignment provided hands-on experience with all core components of Denoising Diffusion Probabilistic Models: the linear noise schedule, the closed-form forward diffusion reparameterisation, a time-conditioned U-Net denoiser, a custom hybrid loss function, and ancestral sampling for image generation. The implementation generates plausible animal images from pure Gaussian noise, confirming that the reverse diffusion chain has been learned correctly.